# POS Project Status

How the project works, what is finished, what is left, and how to make it unique and secure

| 98 | 8 / 15 | 10 | 8 |
| --- | --- | --- | --- |
| API endpoints | areas finished | unique feature ideas | high security fixes |


1. Where the project stands
2. How it fits together
3. Workflows
4. What is left
5. Unique features to build
6. Security
7. Suggested order of work


Prepared 2026-11-01 from the pos-backend code. The backend is what was reviewed: the state of the apps is not covered, so "Done" means done on the server.

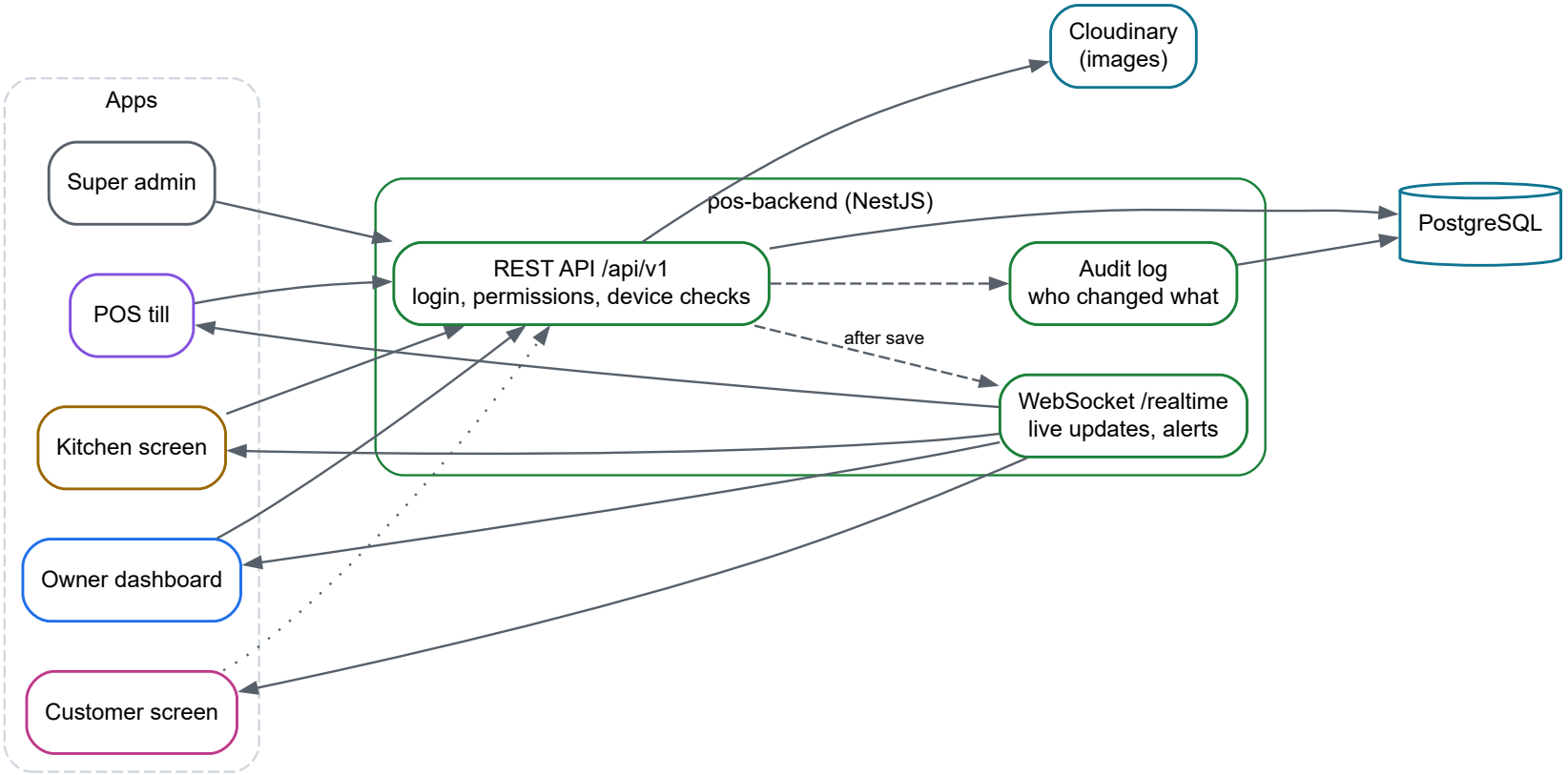
# 1. Where the project stands

The core of a working POS is built: selling, devices, staff, kitchen and customer screens. The main gaps are the business features owners expect daily, and the security fixes needed before real money runs through it.

| Area | Status | Detail |
|---|---|---|
| **Companies, owners, profiles** | Done | Company details, logo, TIN, profile, password and PIN change |
| **Roles and permissions** | Done | Roles, per-worker changes, safe delegation, live updates |
| **Devices** | Done | Approval, types, pairing, per-device MoMo code, activity log |
| **Selling** | Done | Carts, barcode, item notes, split and credit payments |
| **Customer screen** | Done | Instant display, cashier name and photo, MoMo code |
| **Kitchen** | Done | Food-only screen, ready alert, Served button |
| **Floor plan** | Done | Floors, rooms, room types, seats |
| **History** | Done | Audit log and device activity |
| **Stock** | Partly done | Tracking, restock, alerts and expiry done; no suppliers, purchases or stock counts |
| **Reports** | Partly done | Only staff performance exists |
| **Credit sales** | Partly done | Recorded at payment; no list of debts or repayments |
| **Tax** | Partly done | Removed for now; every order has tax 0 |
| **Security** | Partly done | Strong foundations, but section 6 lists fixes needed before launch |
| **Receipts, shifts, refunds, discounts** | Not started | Not started |
| **Offline mode, payment APIs, EBM** | Not started | Not started |

## 2. How it fits together

One server handles every app. Apps change data through the REST API, and the server pushes each change to the right screens over one live connection.

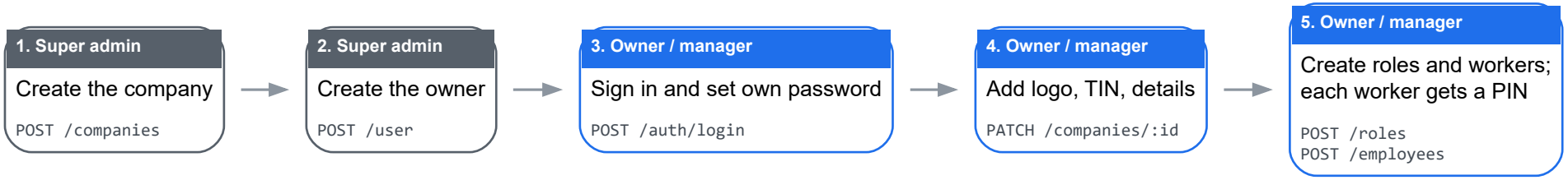


# 3. Workflows

Super admin · Owner / manager · Worker · Device · Server · Customer screen · Kitchen screen

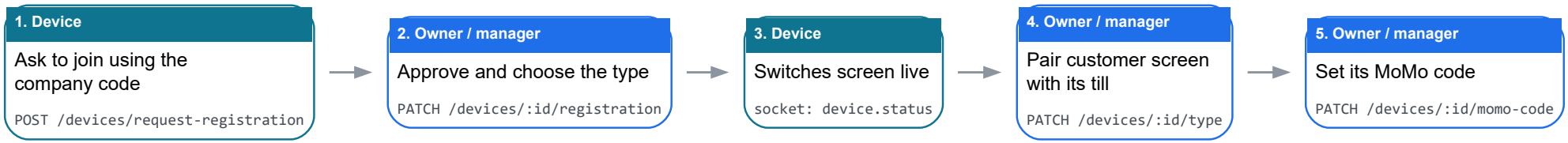
## Starting a business

The platform operator creates each business, so nobody can sign themselves into one.



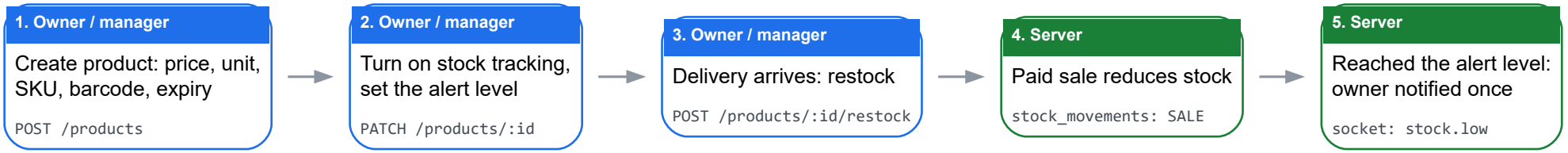

## Setting up devices

Only approved devices work. Each has a job: till, kitchen screen or customer screen.



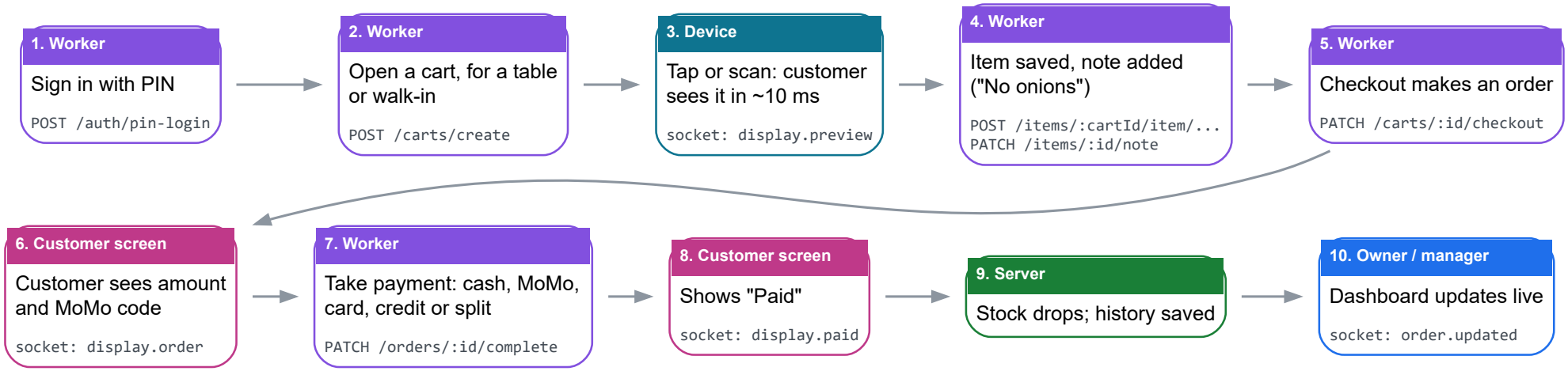

## Products and stock

Stock is optional per product. Every change is logged with who, when and why.



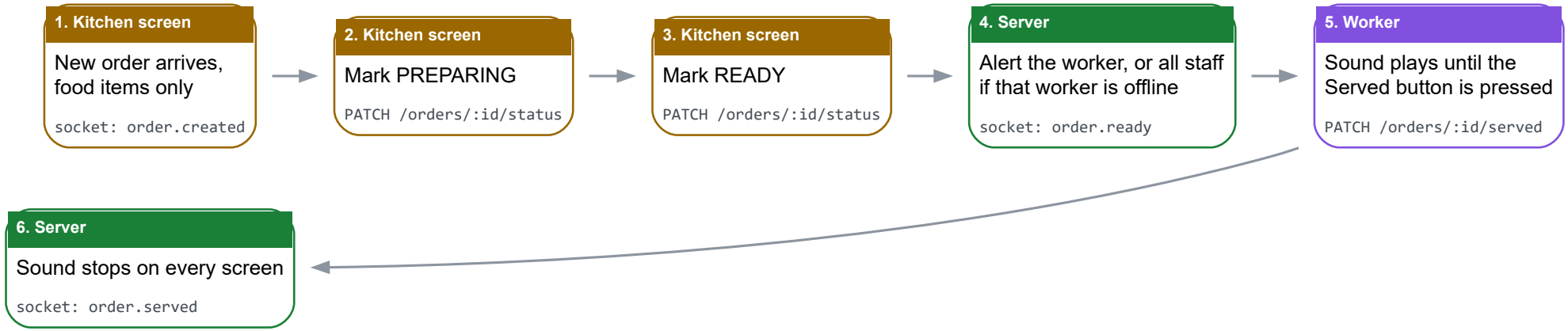
## A sale, from scan to paid

The core of the product. The customer sees each scan at once and the money is checked to the cent.



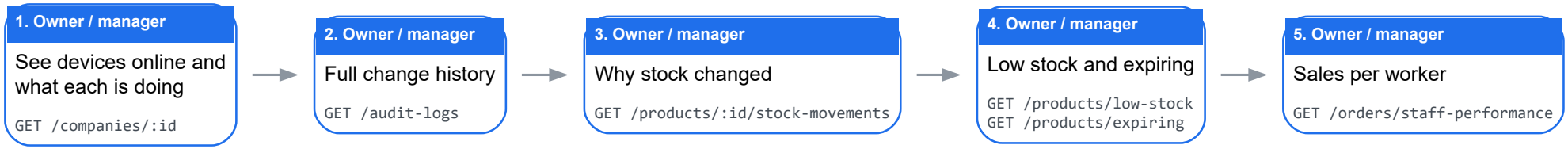

## Kitchen, ready alert and serving

Kitchen screens only get food items. The alert keeps sounding until someone takes the food out.



## Oversight

Owners can always answer "who did this, and when?".



# 4. What is left

## Finish first

| | |
|---|---|
| **Commit the ready alert work** | The kitchen ready alert and Served changes are not committed yet. |
| **Connect the app** | Product fields, restock and stock history, low-stock and expiring lists, stock.low notification, role editor keys, item notes, order.ready sound and Served button, SERVED label everywhere. |
| **Production migrations** | CompanyTin, ProductDetails, ItemNote and OrderServed are on dev only. They run automatically when the server starts with NODE_ENV=production; check the start-up log. |
| **Fix the security items** | Section 6. Do the High items before any real business uses it. |
| **Pay-first shops** | Paid orders are final, so the kitchen cannot mark them READY. Fast-food shops that take payment first need this changed. |
| **Selling by weight** | Cart quantities are whole numbers, so 1.5 kg cannot be sold yet, though stock allows it. |
| **One display per till** | A till can still be paired with several customer screens. Decide if that should be blocked. |
| **Pagination** | GET /orders and GET /carts return every row and will slow down as sales grow. |

## Features still missing

| | |
|---|---|
| **Tax** | Per-item VAT at checkout, saved on each sold item; taxable, exempt and zero-rated. It was built once and removed, so it can return quickly. |
| **Receipts** | Thermal printer receipts and kitchen tickets, plus SMS or WhatsApp receipts. |
| **Reports** | Daily sales, best and worst sellers, profit, payment methods, tax collected, stock value, CSV / PDF export. |
| **Credit book** | Who owes what, since when, repayments and reminders. Permission credits.view is already reserved. |
| **Shifts and cash-up** | Open and close a till, expected against counted cash. Permission finance.verify is reserved. |
| **Refunds and discounts** | Returns linked to the original order, putting stock back. Discounts with a permission. |
| **Fuller stock** | Suppliers, purchase orders, physical stock counts, recipes that use up ingredients. |
| **Forgot password** | There is no reset flow. Email is not set up, so a locked-out owner needs the super admin. |
| **Push notifications** | Sockets only reach open apps, so locked phones miss the ready alert. |
| **Hotel and services** | Reservations, check-in and check-out, room charges. Permission services.manage is reserved. |

# 5. Unique features to build

Ideas that fit Rwanda and are hard for a generic POS to copy. These are suggestions based on your market, not a competitor study, so test them with real shop owners. Offline mode and mobile money are the strongest two.

| Feature | Why it stands out | Effort |
|---|---|---|
| **Works without internet** | The till keeps selling during an outage and syncs later. Internet drops cost shops real money, so this could be your strongest reason to choose this POS. | Hard |
| **MoMo and Airtel Money built in** | The customer gets a payment prompt or scans a QR code on the customer screen, and the order completes itself. No more checking phones for confirmation SMS. | Medium |
| **RRA EBM receipts** | Valid tax receipts directly from the POS. Check current RRA requirements and certification first. | Hard |
| **Kinyarwanda, English, French** | For staff and for each customer screen. Many imported POS systems do not speak Kinyarwanda. | Medium |
| **Daily WhatsApp summary** | The owner gets sales, top products, low stock and anything suspicious each evening, without opening the app. | Medium |
| **Theft and mistake alerts** | Your audit log already has the data: flag many cancellations, price changes before a sale, or stock that does not match. Few small-shop POS systems do this. | Medium |
| **Scan to order at the table** | Customers scan a QR code at their seat, order and pay from their phone. Your floor plan and kitchen screen are ready for it. | Medium |
| **Customer loyalty by phone number** | Points without a card or app, shown on the customer screen, with promotions when the till is idle. | Medium |
| **Credit reminders** | Automatic SMS reminders to customers who owe, a common pain in Rwandan shops. | Low |
| **Multi-branch** | One owner sees every branch live, with stock per branch. Decide early, because stock would move to its own table. | Hard |

# 6. Security

## Already in place

- Passwords and PINs hashed with bcrypt
- Device secrets random, stored hashed, compared safely
- Permissions read live and refused by default
- No staff escalation: nobody gives access they lack
- Every query limited to the user's company
- Workers see only their own carts and orders
- Single-device sessions; instant sign-out on deactivate
- Only approved devices work; disable ends sessions
- Login lock-out after 5 failures
- Unknown input fields rejected on every request
- Upload type and 10 MB size limits
- Audit log with secrets masked
- Customer screens get only safe fields
- Payments checked by the server to the cent
- Paying twice is blocked by a database lock
- Error stacks hidden in production

## Before launch

- HTTPS only, with the database not open to the internet
- Different secrets for production; change them if ever shared
- Automatic daily backups, and one test restore
- Error tracking and alerts (e.g. Sentry)
- Rotate the JWT secret on a schedule
- Keep the Cloudinary and database keys out of the app and repo
- Delete old test accounts and data
- Write a simple plan for a stolen device or leaked password
- Rwanda data protection law: review how you store customer and staff data
- Independent penetration test before large customers

# 6. Security: what to fix

Found by reviewing the current code. This is a code review, not a penetration test, so it may not cover everything.

| Level | Problem | Where | Risk | Fix |
|---|---|---|---|---|
| High | **Vulnerable packages** | `package.json` | npm audit reports 8 known vulnerabilities: critical in proxy-addr; high in multer (your uploads), body-parser and js-yaml; moderate in qs and typeorm. | Run npm audit fix, retest, and check npm audit on every release. |
| High | **PIN login does not need the device secret** | `auth.service.ts pinLogin` | Anyone who learns a till's device id can try PINs from any computer. The secret exists but login never checks it. | Send x-device-secret with PIN login and refuse without it. |
| High | **One public request can tie up the server** | `auth.service.ts pinLogin` | Each wrong PIN runs bcrypt against every employee in the company, with no sign-in needed. A few requests per second can slow all tills. | Add the device secret check and a request limit. Better: look up the worker first (name or ID + PIN). |
| High | **Anyone can lock a till** | `login-throttle.service.ts` | Five wrong PINs lock the whole device for 15 minutes, and five wrong passwords lock an owner. An attacker can do this on purpose. | Limit by device and IP together; use short, growing delays. |
| High | **Logging out does not cancel the token** | `auth.service.ts logout` | A copied token keeps working for up to a day after logout. | Clear the session on logout and check it for every role. Add short tokens with refresh tokens. |
| High | **PINs and company codes use Math.random** | `user.service.ts:389, company.entity.ts:83` | Math.random is predictable and not meant for secrets. | Use crypto.randomInt for PINs and crypto.randomBytes for codes. |
| High | **No request limits on public endpoints** | `main.ts` | Only login attempts are limited. Registration, verify and every other route can be flooded. | Add @nestjs/throttler globally, stricter on public routes. |
| High | **Any website can call the API** | `main.ts enableCors, gateway cors: *` | CORS allows every origin on REST and sockets. | Allow only your app domains, from an environment setting. |
| Medium | **Forced password change is not enforced** | `mustChangePassword` | The flag is sent to the app, but the server still lets the old password work everywhere. | Refuse other endpoints until the password is changed. |
| Medium | **Swagger is public** | `main.ts` | The full API map is open in production. | Turn it off in production or put a password on it. |
| Medium | **No security headers** | `main.ts` | Browsers get no extra protection. | Add helmet. |
| Medium | **No second step for powerful accounts** | `Super admin, owners` | One stolen password opens every company. | Add two-factor login, first for the super admin. |
| Medium | **Internal error text can reach users** | `all.exceptions.filter.ts` | Unexpected errors return exception.message in production. | Return a generic message and only log the detail. |
| Medium | **Upload checks trust the browser** | `file-upload.interceptor.ts` | The file type comes from the client, and the extension from the file name. | Check the file's real content, and pick the extension from it. |

| Level | Problem | Where | Risk | Fix |
| --- | --- | --- | --- | --- |
| Medium | **Open sockets outlive the token** | `realtime.gateway.ts` | A socket checks the token only when it connects, so it can stay open after expiry. | Disconnect when the token expires. |
| Medium | **Weak password rules** | `change-password.dto.ts` | Only a minimum of 8 characters, and no maximum. | Add a maximum of 72 and refuse common passwords. |
| Low | **Socket preview is not rate limited** | `realtime.gateway.ts` | A signed-in till could flood its customer screen. | Limit events per socket. |
| Low | **No automated checks** | `Repository` | There is no CI, and the end-to-end tests are not in the repo. | Add them to test/ and run them on every push. |

## 7. Suggested order of work

### Now: make it safe

- Commit current work
- npm audit fix
- Device secret on PIN login
- Crypto-safe PINs and codes
- Logout cancels the session
- Rate limits, CORS, helmet
- Hide Swagger in production

### Next: launch ready

- Connect all app screens
- Same-region hosting
- Pagination
- Receipts
- Tax
- Backups, monitoring, CI

### Then: run a business

- Reports
- Credit book
- Shifts and cash-up
- Refunds and discounts
- Forgot password, 2FA

### Later: stand out

- MoMo and Airtel APIs
- Offline mode
- EBM receipts
- Languages
- WhatsApp summaries
- Theft alerts